NAME:M.RAMAKRISHNA RAJU
STUDENT ID:2400031163

DEPARTMENT OF CSE, CS&IT, AI&DS
COURSE CODE: 24SDCS02E FULL STACK APPLICATION DEVELOPMENT
Date of the Session:6-01-2026
#SKILL - 2 Hibernate CRUD Operations
Prerequisites:
 Basic knowledge of Java
 Understanding of classes, objects, and OOP
 Familiarity with SQL basics

A retail inventory system needs to store product details such as **productId**, **name**, **description**, **price**,
and **quantity**. The admin should be able to **add new products**, **retrieve product information**, **update**
price or quantity, and **delete discontinued products**.
Your task is to **implement complete CRUD operations** using **Hibernate** with proper entity mapping
using **JPA annotations**.
1. Create a **Product entity** with fields: **id (primary key), name, description, price, quantity**.
2. Configure **Hibernate** and map the Product entity to a table using **JPA annotations**.
3. Insert **multiple Product records** into the database.
4. Retrieve a product using its **id**.
5. Update the **price** or **quantity** of any selected product.
6. Delete a product record by **id** if it is discontinued.
7. **Note:**
a. Implement Hibernate **ID generation strategies** (**AUTO, IDENTITY, SEQUENCE**)
inside the entity to observe how primary key values differ.
b. The **ID can be manual or auto generated**, based on your chosen strategy.
8. Push the Hibernate project into a single GitHub repository.

Structure:

Codes:

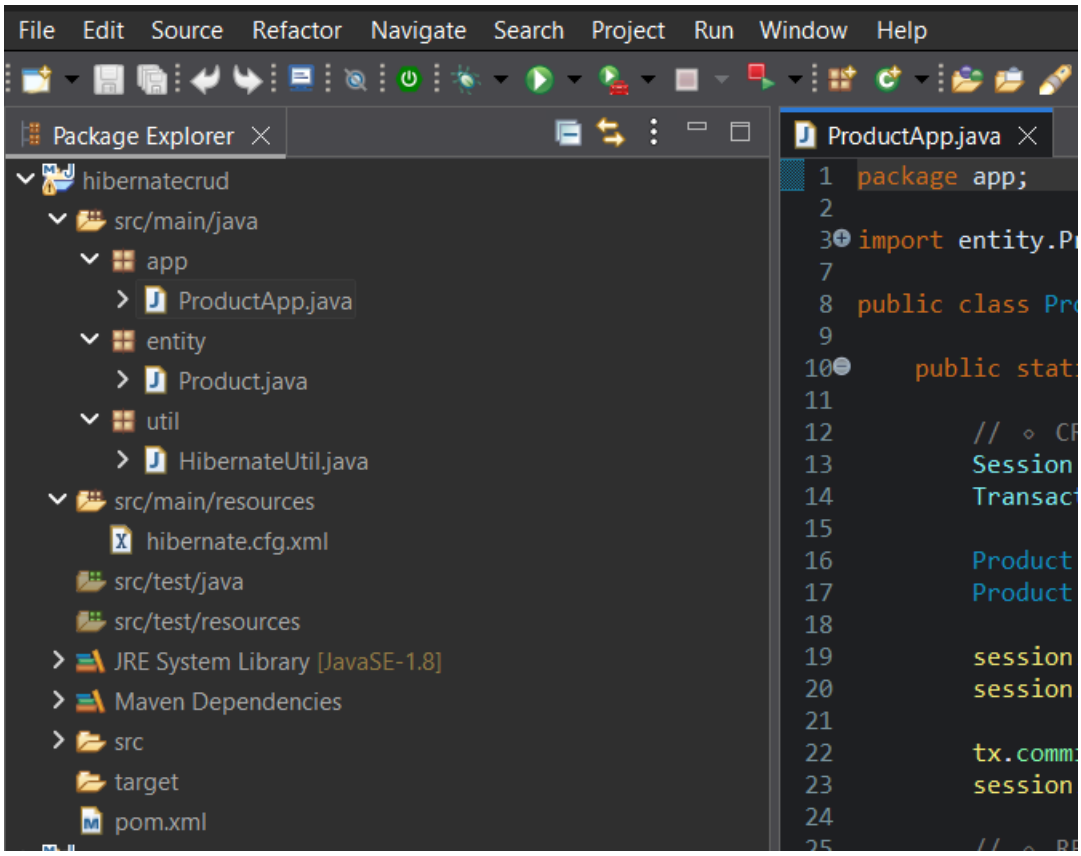
ProductApp.java

```
package app;

import entity.Product;
import org.hibernate.Session;
import org.hibernate.Transaction;
import util.HibernateUtil;

public class ProductApp {
```

```
public static void main(String[] args) {
```




```
    // 🔹 CREATE
    Session session = HibernateUtil.getSessionFactory().openSession();
    Transaction tx = session.beginTransaction();

    Product p1 = new Product("Laptop", "Gaming Laptop", 75000, 10);
    Product p2 = new Product("Mouse", "Wireless Mouse", 1200, 50);

    session.save(p1);
    session.save(p2);

    tx.commit();
    session.close();

    // 🔹 READ
    session = HibernateUtil.getSessionFactory().openSession();
    Product product = session.get(Product.class, 1);
    System.out.println("Product Name: " + product.getName());
    session.close();

    // 🔹 UPDATE
    session = HibernateUtil.getSessionFactory().openSession();
    tx = session.beginTransaction();

    product = session.get(Product.class, 1);
    product.setPrice(70000);
```

```
        product.setQuantity(8);

        session.update(product);
        tx.commit();
        session.close();

        // 🔹 DELETE
        session = HibernateUtil.getSessionFactory().openSession();
        tx = session.beginTransaction();

        Product deleteProduct = session.get(Product.class, 2);
        session.delete(deleteProduct);

        tx.commit();
        session.close();

        System.out.println("CRUD operations completed successfully.");
    }
}
```

Product.java:

```
package entity;

import javax.persistence.*;

@Entity
@Table(name = "product")
public class Product {

@Id
// 🔁 Change strategy to AUTO / IDENTITY / SEQUENCE to observe difference
@GeneratedValue(strategy = GenerationType.IDENTITY)
private int id;

private String name;
private String description;
private double price;
private int quantity;

// Constructors
public Product() {}

public Product(String name, String description, double price, int quantity) {
this.name = name;
this.description = description;
this.price = price;
this.quantity = quantity;
}

// Getters & Setters
public int getId() { return id; }

public String getName() { return name; }
```

```
public void setName(String name) { this.name = name; }

public String getDescription() { return description; }
public void setDescription(String description) { this.description = description; }

public double getPrice() { return price; }
public void setPrice(double price) { this.price = price; }

public int getQuantity() { return quantity; }
public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

HibernateUtil.java:

```
package util;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

    private static SessionFactory factory;

    static {
        factory = new Configuration()
                .configure("hibernate.cfg.xml")
                .buildSessionFactory();
    }

    public static SessionFactory getSessionFactory() {
        return factory;
    }
}
```

Hibernate.cfg,xml:

```
<hibernate-configuration>
<session-factory>

<property name="hibernate.connection.driver_class">
com.mysql.cj.jdbc.Driver
</property>

<property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/db1
</property>

<property name="hibernate.connection.username">root</property>
<property name="hibernate.connection.password">Mahesh@31164</property>

<property name="hibernate.dialect">
org.hibernate.dialect.MySQL8Dialect
</property>

<property name="hibernate.hbm2ddl.auto">update</property>
<property name="hibernate.show_sql">true</property>
```

```
<mapping class="entity.Product"/>

</session-factory>
</hibernate-configuration>
```

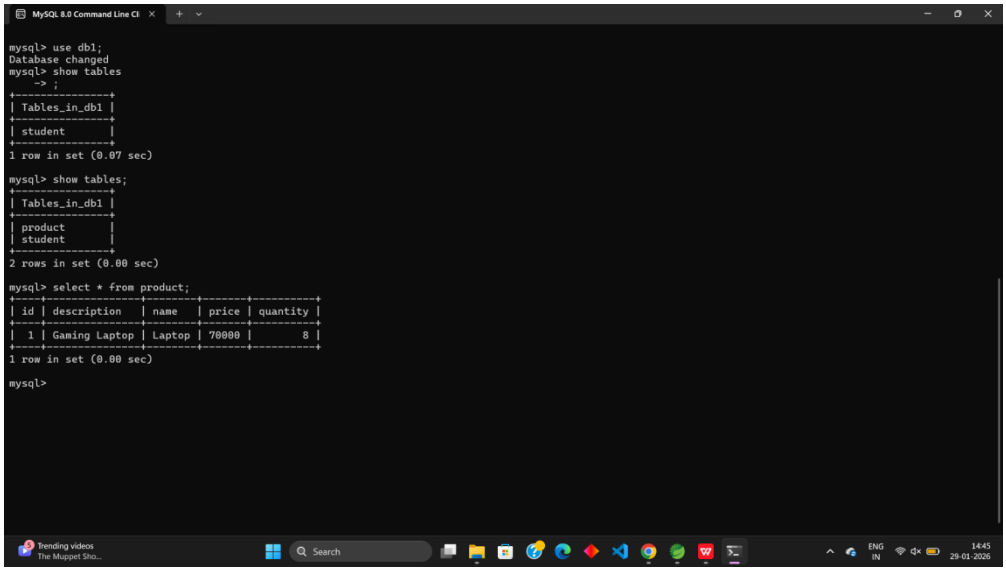
pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>org.hibernate</groupId>
<artifactId>hibernatecrud</artifactId>
<version>0.0.1-SNAPSHOT</version>
<dependencies>
<!-- Hibernate Core -->
<dependency>
<groupId>org.hibernate</groupId>
<artifactId>hibernate-core</artifactId>
<version>5.6.15.Final</version>
</dependency>

<!-- MySQL Connector -->
<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>8.0.33</version>
</dependency>

<!-- JPA API -->
<dependency>
<groupId>javax.persistence</groupId>
<artifactId>javax.persistence-api</artifactId>
<version>2.2</version>
</dependency>
</dependencies>
</project>
```

OutPut:



```
mysql> use db1;
Database changed
mysql> show tables
    -> ;
+---------------+
| Tables_in_db1 |
+---------------+
| student       |
+---------------+
1 row in set (0.07 sec)

mysql> show tables;
+---------------+
| Tables_in_db1 |
+---------------+
| product       |
| student       |
+---------------+
2 rows in set (0.00 sec)

mysql> select * from product;
+----+---------------+--------+-------+----------+
| id | description   | name   | price | quantity |
+----+---------------+--------+-------+----------+
|  1 | Gaming Laptop | Laptop | 70000 |        8 |
+----+---------------+--------+-------+----------+
1 row in set (0.00 sec)

mysql>
```

VIVA-VOCE:

1. What is ORM and why is it used in application development?

Ans: **ORM (Object Relational Mapping)** is a technique that allows developers to **map Java objects to database tables** and vice versa.

Why ORM is used:

- Eliminates writing **complex SQL queries**
- Enables **database-independent** code

2. Which annotation is used to mark a class as a JPA entity?

Ans: @Entity

3. Why must every Hibernate entity have a primary key?

Ans: Every Hibernate entity **must have a primary key** because:

- Hibernate uses it to **uniquely identify** each object
- Required for **CRUD operations**

4. How do you update an existing record using Hibernate?

Ans: Method 1: Update using update()

- Student s = session.get(Student.class, 1);
- s.setName("Updated Name");
- session.update(s);

Method 2: Update using merge()

session.merge(studentObject);

5. What is the difference between get() and load() in Hibernate?
Ans:

Feature	get()	Load()
Data base hit	Actual object	Proxy object
Use case	When data may not exist	When data is sure to exist